

OpenViking Relations 实验报告

1. 术语规范

在本次实验中，我们统一以下术语：

术语	含义	对应配置标志
agentic	Agent 循环驱动：Agent 自主决定搜索、阅读、链接，最多迭代 N 轮后生成答案	<code>use_vikingbot: true</code>
unagentic	传统 RAG 流水线：向量检索 → 拼接上下文 → LLM 直接生成答案	<code>use_vikingbot: false</code>
learned	使用 <code>.relations.jsonl</code> 关系数据增强检索	<code>use_relations: true</code>
unlearned	纯向量检索，不使用关系数据	<code>use_relations: false</code> 或未设置

四种实验模式：

模式	描述	典型配置
agentic-learned	VikingBot Agent + 搜索时查询 relations	<code>*_bot_config_relations.yaml</code>
agentic-unlearned	VikingBot Agent 纯搜索	<code>*_bot_config.yaml</code>
unagentic-learned	标准 RAG 向量检索 + 搜索时查询 relations	<code>*_config_relations.yaml</code>
unagentic-unlearned	标准 RAG 纯向量检索	<code>*_config.yaml</code>

2. 实验方法

2.1 Relations 的创建

Relations 由 agentic 模式下的 **build-links** 流程创建。当 `enable_linking: true` 时，Agent (bot)完成回答后执行 `_post_answer_link()` 方法。

创建流程：

1. Agent 完成问答循环（搜索 → 阅读 → 回答）

- 2. 收集本次循环中所有被 `openviking_read / openviking_multi_read` 成功读取的文档 URI 列表
- 3. 对 URI 列表中每对不同文档之间都创建一条双向关系(消融不同轮次的创立链接)
- 4. 调用 `VikingClient.link(from_uri, to_uri, query=原始问题)`
- 5. 同时写入 `{from_uri 父目录}/.relations.jsonl, {to_uri 父目录}/.relations.jsonl`

每条 relation 记录包含：

字段	说明	示例
<code>uri1</code>	源文档 URI	<code>viking://resources/HotpotQA_processed_docs/.../doc_001.md</code>
<code>uri2</code>	目标文档 URI	<code>viking://resources/HotpotQA_processed_docs/.../doc_023.md</code>
<code>query_question</code>	触发链接的原始用户问题	<code>"Which company acquired DeepMind in 2014?"</code>
<code>query_embedding</code>	问题的向量表示（由豆包 Embedding 模型生成）	<code>[0.0123, -0.0456, ...]</code> (1024 维)(<u>转化为id, 可以减少存储消耗</u>) (<u>uri1,uri2聚合</u>)

创建方式的关键特征：

- **问题驱动**：每条 relation 绑定了一个具体问题主题。"当回答与问题 X 相关主题的问题时，这两篇文档被一起阅读了"
- **双向写入**：`uri1 → uri2` 和 `uri2 → uri1` 各写一条，支持从任一方向查询

2.2 Relations 的搜索与使用

unagentic-learned 模式：向量检索 + Relations 增强

在 `VikingStoreWithRelations.retrieve()` 中
(`src/core/vector_store_with_relations.py`)：

- 1. **向量检索**：对原始 query 做向量搜索，取 top-k 结果
- 2. **Relations 查询**：对 top-5 结果的每个 URI，读取其目录下的 `.relations.jsonl`
- 3. **双路匹配**（对每条 relation 记录执行）：
 - **关键词匹配 (Path 1)**：提取当前 query 和记录中 `query_question` 的关键词（分词 + 去停用词），计算关键词覆盖度。若当前 query 的关键词在历史关键词中覆盖 > 70%，则命中
 - **向量匹配 (Path 2)**：计算当前 query 的 embedding 与记录中 `query_embedding` 的余弦相似度。若相似度 > 0.7，则命中
 - 两路为 **OR 关系**：任一路命中即采纳
- 4. **追加结果**：命中的关联文档追加到上下文中，供 LLM 生成答案时参考



agentic-learned 模式：Agent 搜索时内嵌 Relations 查询

在 `vikingSearchTool.execute()` 中 (`bot/vikingbot/agent/tools/ov_file.py`)：

- 1. **搜索**：Agent 调用 `openviking_search` 工具
- 2. **Relations 查询**：对每个搜索结果，调用 `search_client.relations(uri, query=query)`
- 3. **结果展示**：命中的关联文档以 `[via relations] uri` 形式追加到搜索结果中
- 4. **Agent 决策**：Agent 看到 `[via relations]` 标记的文档后，自行决定是否使用 `openviking_read` 阅读

```
Agent: openviking_search("waterloo battle")
↓
搜索结果: [doc1, doc2, doc3, doc4, doc5]
↓
对每个结果查 relations(doc_i, "waterloo battle")
↓
doc1 的关联: [doc_a, doc_b] ← 历史问答中与 doc1 一起被读过的文档
doc3 的关联: [doc_c]
↓
搜索结果展示:
1. doc1
2. doc2
3. doc3
...
[via relations] doc_a
[via relations] doc_b
[via relations] doc_c
↓
Agent 决定: openviking_multi_read([doc1, doc_a, doc_b])
```

2.3 两种模式使用 Relations 的关键区别

维度	unagentic-learned	agentic-learned
谁决策使用	系统自动追加到 context	Agent 看到标记后自主决定
追加方式	直接拼入 LLM prompt 的 context	展示在搜索结果中，Agent 选择是否阅读
匹配方式	关键词 + 向量双路匹配	同左（调用的都是 <code>vikingClient.relations()</code> ）
Relations 暴露量	匹配的文档全部追加	Agent 可根据相关性筛选

2.4 关键词匹配算法详解

整个关键词匹配流程分为两个阶段：**关键词提取** 和 **覆盖率匹配**。代码位于 `bot/vikingbot/openviking_mount/ov_server.py` 的 `_extract_keywords()` 和 `relations()` 方法中。

2.4.1 关键词提取: `_extract_keywords(text)`

```
_ENGLISH_STOPWORDS = {
    "a", "an", "the", "and", "or", "but", "in", "on", "at", "to", "for",
    "of", "with", "by", "from", "is", "it", "as", "be", "was", "were",
    "been", "are", "am", "do", "did", "does", "has", "had", "have", "will",
    "would", "could", "should", "may", "might", "shall", "can", "not", "no",
    "nor", "so", "if", "then", "than", "that", "this", "these", "those",
    "what", "which", "who", "whom", "how", "when", "where", "why",
    "all", "each", "every", "both", "few", "more", "most", "other", "some",
    "such", "only", "own", "same", "too", "very", "just", "about", "above",
    "after", "again", "also", "any", "because", "before", "below", "between",
    "during", "into", "its", "out", "over", "through", "under", "until",
    "up", "down", "here", "there", "once", "further",
    ... // ~100 个英文停用词
}

def _extract_keywords(text: str) -> set:
    if not text:
        return set()
    tokens = text.lower().split()          # Step 1: 小写化 + 空格分词
    result = set()
    for t in tokens:
        t = t.strip(".,;:!?\"'()[]{}---") # Step 2: 去掉首尾标点
        if len(t) <= 2 or t in _ENGLISH_STOPWORDS: # Step 3: 过滤短词和停用词
            continue
        result.add(t)
    return result
```

3. 实验流程

3.1 执行步骤

实验分三个阶段，按顺序执行：

Phase 1 — Unlearned 基线收集

先跑两组不用 relations 的基线，获得对比基准：

```
python run.py --config config/{dataset}/{dataset}_bot_config.yaml #
agentic-unlearned
python run.py --config config/{dataset}/{dataset}_config.yaml #
unagentic-unlearned
```

这两组配置中 `use_relations` 均为 `false`（或未设置），完全依赖向量检索。

Phase 2 — 构建 Relations

跑 `agentic + build-links` 模式，让 Agent 在回答问题的过程中创建文档间的关系链：

```
python run.py --config config/{dataset}/{dataset}_bot_config_build_links.yaml #
agentic-build-links
```

关键配置：

- `use_vikingbot: true` — 使用 Agent
- `enable_linking: true` — 开启链接功能
- 每个问题回答完毕后, `_post_answer_link` 将所有被读文档两两链接
- 结果写入各文档目录下的 `.relations.jsonl`

Phase 3 — Learned 评估

Relations 数据就绪后, 跑 learned 模式评估效果:

```
python run.py --config config/{dataset}/{dataset}_bot_config_relations.yaml      #  
agentic-learned  
python run.py --config config/{dataset}/{dataset}_config_relations.yaml      #  
unagentic-learned
```

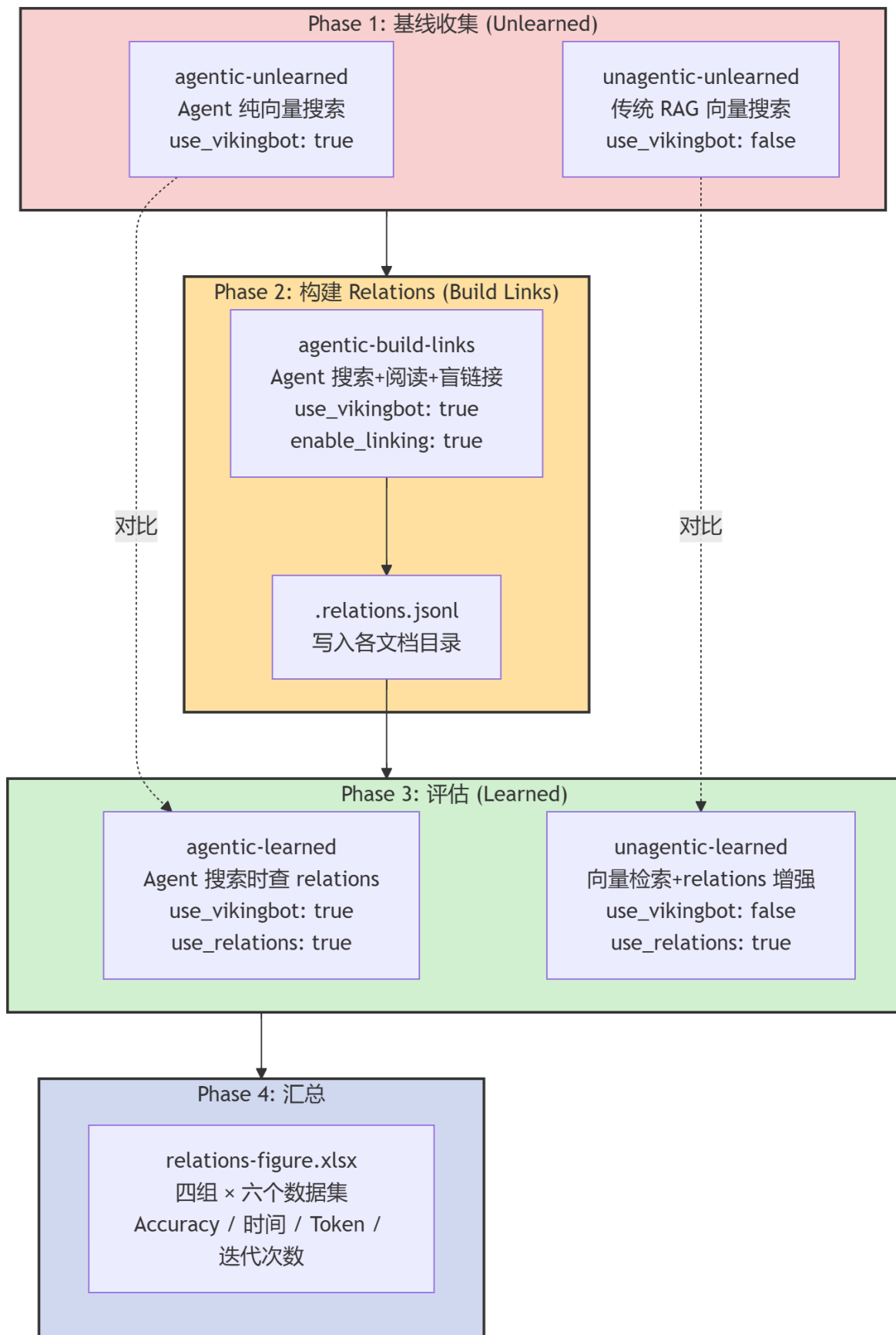
关键配置:

- `use_relations: true` — 启用 relations 增强检索
- `embedding` 配置段 — 提供向量匹配所需的 Embedding 模型

Phase 4 — 汇总数据

将四组实验的指标 (Accuracy、检索时间、Token 成本、迭代次数) 汇总到 `relations-figure.xlsx`。

3.2 流程图



Syntax error in graph
mermaid version 8.14.0

实验设计的核心思路:

对比 learned vs unlearned 在两种架构 (agentic / unagentic) 下的差异:

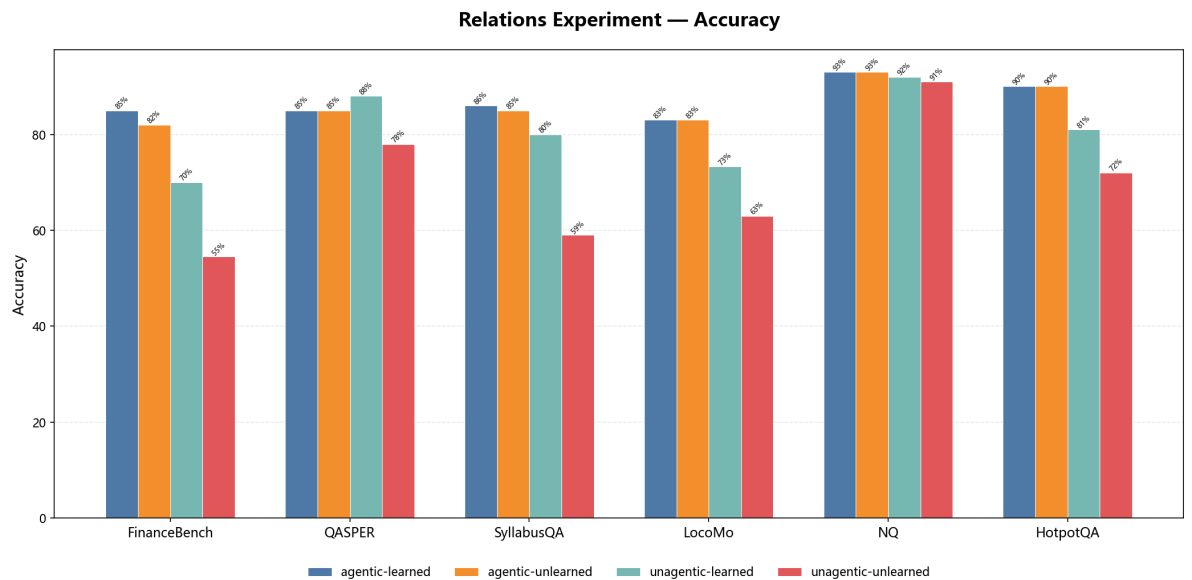
- **agentic-learned vs agentic-unlearned**: Relations 能否帮助 Agent 在更少的轮次发现更多相关文档?
- **unagentic-learned vs unagentic-unlearned**: Relations 增强的检索能否提升传统 RAG 的召回和准确率?

4. 实验结果

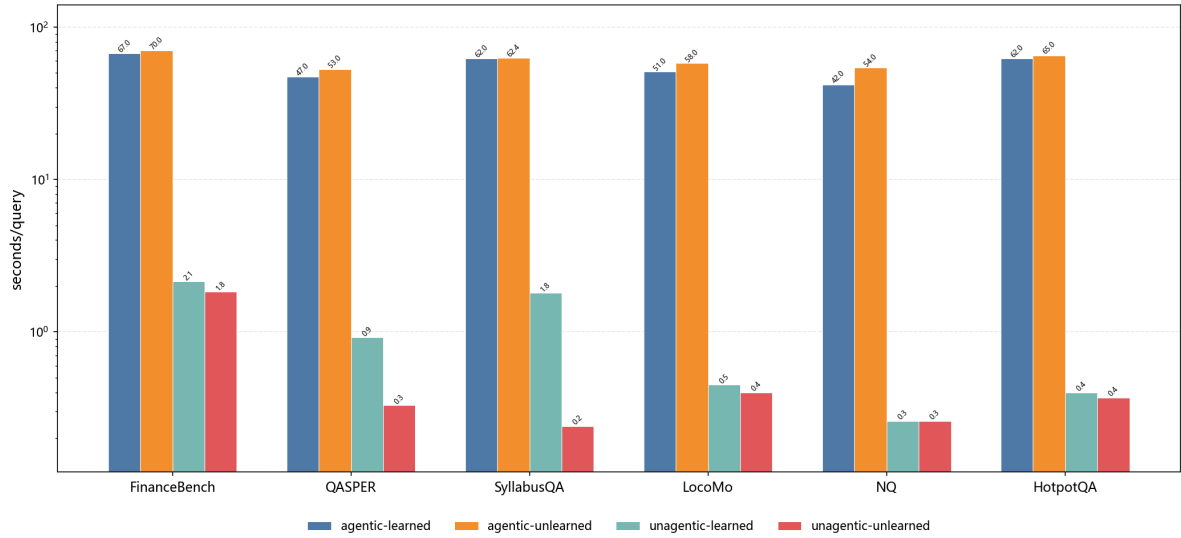
4.1 指标说明

指标	包含字段	刻度	含义
Accuracy	accuracy	线性 0-100	LLM-as-judge 评分 (0-4 分归一化)
检索时间	时间	对数	每 query 平均检索耗时 (秒), 含向量搜索 + relations 查询
检索 Token 成本	token	对数	每 query 消耗的 token 数。格式 输入+输出=总计, 取等号后总 token 数
迭代次数	iterations	线性	agentic 模式平均迭代轮数 (unagentic 模式固定为 1)

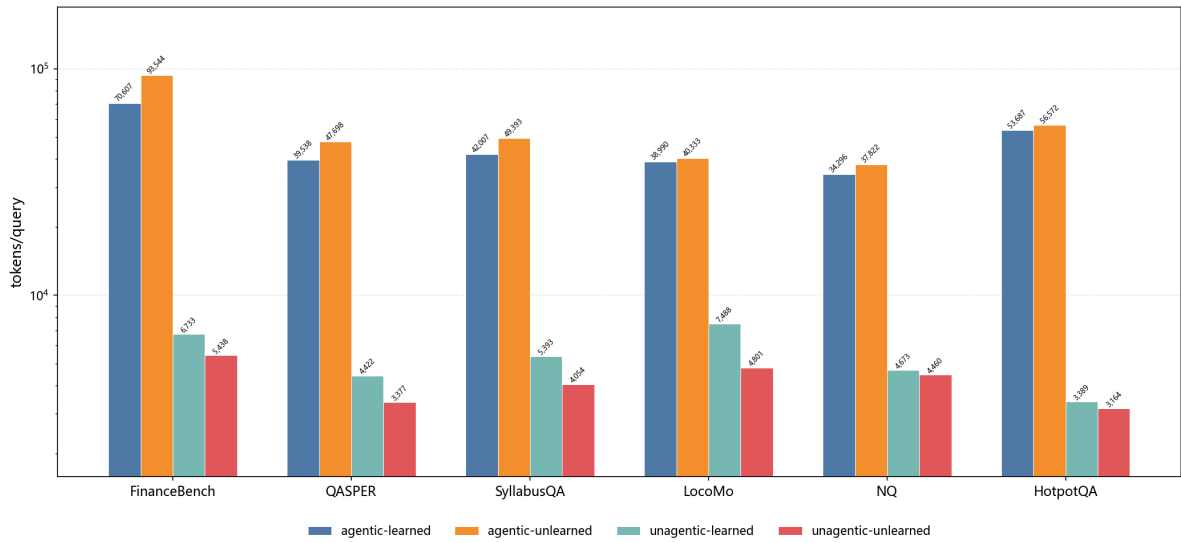
4.2 图表



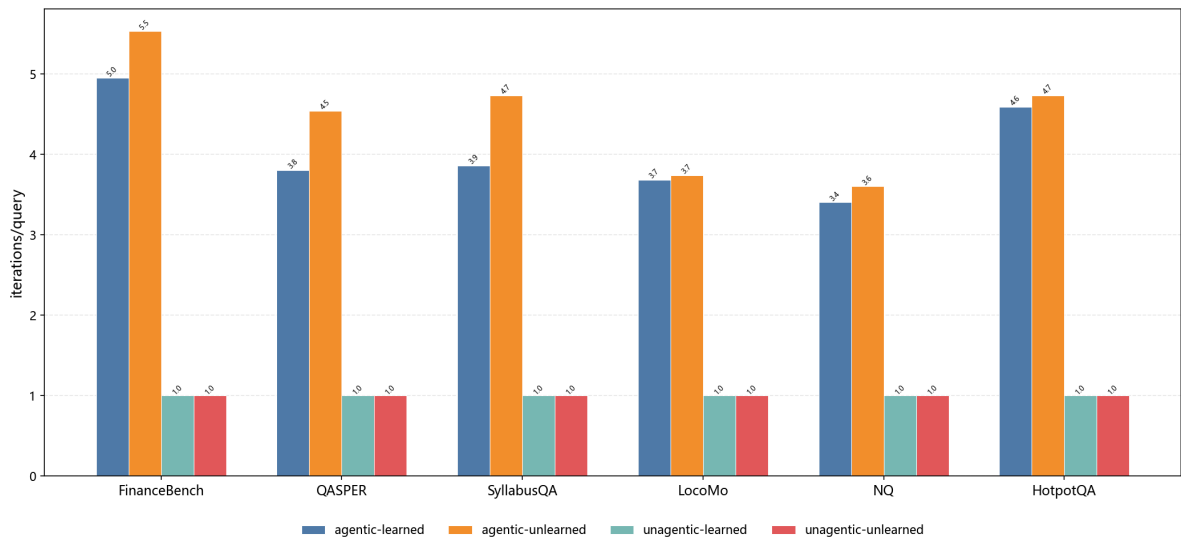
Relations Experiment — 检索时间



Relations Experiment — 检索Token成本



Relations Experiment — 迭代次数



4.3 数据表

agentic-learned

数据集	Accuracy	检索时间	Token 成本	迭代次数
FinanceBench	85%	67s	70,607	4.95
QASPER	85%	47s	39,538	3.80
SyllabusQA	86%	62s	42,007	3.86
LocoMo	83%	51s	38,990	3.68
NQ	93%	42s	34,296	3.40
HotpotQA	90%	62s	53,687	4.59

agentic-unlearned

数据集	Accuracy	检索时间	Token 成本	迭代次数
FinanceBench	82%	70s	93,544	5.53
QASPER	85%	53s	47,698	4.54
SyllabusQA	85%	62s	49,393	4.73
LocoMo	83%	58s	40,333	3.74
NQ	93%	54s	37,822	3.60
HotpotQA	90%	65s	56,572	4.73

unagentic-learned

数据集	Accuracy	检索时间	Token 成本	迭代次数
FinanceBench	70%	2.14s	6,733	1
QASPER	88%	0.92s	4,422	1
SyllabusQA	80%	1.80s	5,393	1
LocoMo	73%	0.45s	7,488	1
NQ	92%	0.26s	4,673	1
HotpotQA	81%	0.40s	3,389	1

unagentic-unlearned

数据集	Accuracy	检索时间	Token 成本	迭代次数
FinanceBench	54%	1.83s	5,438	1
QASPER	78%	0.33s	3,377	1
SyllabusQA	59%	0.24s	4,054	1

数据集 Locomo	Accuracy 63%	检索时间 0.40s	Token 成本 4,801	迭代次数 1
NQ	91%	0.26s	4,460	1
HotpotQA	72%	0.37s	3,164	1

4.4数据结论

- 1. **learned vs unlearned 在 agentic 模式中轮次降低准确率升高**: Accuracy 在 6 个数据集上略有升高 ($\pm 3\%$) , learned 的迭代轮次更少, Token 成本更低,
- 2. **learned 在 unagentic 模式中有显著提升**: FinanceBench 从 54% $\rightarrow$ 70% (+16%) , SyllabusQA 从 59% $\rightarrow$ 80% (+21%) , 因为 relations 直接扩展了 LLM 的 context 结果统计性的量化的数据

5. 有效 Case 分析

Case 1: unagentic search增强

问题:

What will happen to my grade if I make all of my posts but never respond to anyone else's posts on the forum? (如果我只发帖而不回复论坛上其他人的帖子, 我的成绩会受到什么影响?)

证据:

Grade_Component_Percent_of_3more.md

unagentic模式检索结果:

- unagentic-learned
 - search
 - "viking://resources/SyllabusQA_processed_docsBIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/UMass_and_BIOCHEM_320_Course_Policies.md"
 - "viking://resources/SyllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/BIOCHEM_320_Class_Schedule_Spring_2023.md"
 - "viking://resources/SyllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/BIOCHEM_320_Syllabus_SP23__6more.md"
 - "viking://resources/SyllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/Course_Communication_Student_Hours_and_Ot_f886f1a7.md"
 - "viking://resources/SyllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/Assignments_and_Graded_Components/Homework-Mastering_Chemi_4more.md"
 - relations
 - "viking://resources/SyllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/Assignments_and_Graded_Components/.abstract.md",
 - "viking://resources/SyllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/Assignments_and_Graded_Components/Grade_Component_Percent_of_3more.md"

分析:

unagentic通过relations召回了普通向量召回不到的内容。

Case 2: agentic缩短检索轮次

问题:

Is attendance graded on the first day? (第一天的出勤率会评分吗?)

证据:

BIOCHEM_320_Syllabus_SP23_2_Feb_2

023_doc/BIOCHEM_320_Class_Schedule_Spring_2023.md"

- agentic-learned

```
"tool_calls": [  
  {  
    "tool_name": "openviking_search",  
    "args": {  
      "query": "BIOCHEM 320 Syllabus SP23 2 Feb 2023",  
      "target_uri": "viking://resources/"  
    },  
    "relations_found": 2  
  },  
  {  
    "tool_name": "openviking_multi_read",  
    "args": {  
      "uris": [  
        "BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/Assignments_and_Graded_Components/Grade_Component_Percent_of_3more.md",  
        "BIOCHEM_320_Syllabus_SP23_2_Feb_2023_doc/BIOCHEM_320_Class_Schedule_Spring_2023.md"  
      ]  
    },  
    "relations_found": 0  
  }  
]
```

- agentic-unlearned

```
"tool_calls": [  
  {  
    "tool_name": "openviking_search",  
    "args": {  
      "query": "BIOCHEM 320 Syllabus SP23 2 Feb 2023",  
      "target_uri": "viking://resources/"  
    },  
    "relations_found": 0  
  },  
  {  
    "tool_name": "list_dir",  
    "args": {  

```

```

        "path": "BIOCHEM_320_Syllabus_SP23_2_Feb_2
023_doc/Assignments_and_Graded_Components/"
    },
    "relations_found": 0
},
{
    "tool_name": "openviking_multi_read",
    "args": {
        "uris": [
            "BIOCHEM_320_Syllabus_SP23_2_Feb_
2023_doc/BIOCHEM_320_Syllabus_SP23__6more.md",
            "BIOCHEM_320_Syllabus_SP23_2_Feb_2
023_doc/Assignments_and_Graded_Components/.abstract.md"
        ]
    },
    "relations_found": 0
},
{
    "tool_name": "list_dir",
    "args": {
        "path":
"syllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2 023_doc/"
    },
    "relations_found": 0
},
{
    "tool_name": "openviking_grep",
    "args": {
        "uri":
"viking://resources/syllabusQA_processed_docs/BIOCHEM_320_Syllabus_SP23_2_Feb_2
023_doc/",
        "pattern": [
            "attendance",
            "graded",
            "first day"
        ]
    },
    "relations_found": 0
},
{
    "tool_name": "openviking_multi_read",
    "args": {
        "uris": [
            "BIOCHEM_320_Syllabus_SP23_2_Feb_
2023_doc/Assignments_and_Graded_Components/Grade_Component_Percent_of_3more.md",
            "BIOCHEM_320_Syllabus_SP23_2_Feb_2
023_doc/BIOCHEM_320_Class_Schedule_Spring_2023.md"
        ]
    },
    "relations_found": 0
}
],

```

分析:

agentic在第一轮search时触发relations搜索，根据日志"relations_found": 2得出查找到2个相关文档，相关文档使bot，在第一轮search即查询到结果，有效缩短了查询流程。

6. 总结

1. **Relations 可以有效的降低Agentic的迭代轮次**
2. **Relations 对 unagentic 模式的提升最明显**，因为直接扩展了 LLM 的输入上下文
3. **后续优化方向**
 - 链接阈值：为向量匹配和关键词匹配设置更严格的阈值
 - 链接衰减：旧链接随时间降低权重，如果长时间不访问则删除这个链接
4. **我尝试过让bot自己决定两个文档是否真的相关，但是效果不好**